

Test Strategy Alignment Review

Are the team's testing methods and QA's strategy pointing the same way? Reviewed against everything merged between 17 and 21 September — four new application pipelines and eight new Terraform modules.

AUTHOR	REVIEWED	RANGE	DAYS TO SUBMIT
Shanthi Subramanian · BA / QA	21 September 2026	9106553...4e36778 a816378...bc45271	9

62 OF 80 TESTS IN CI	1 OF 4 QA LAYERS GATED	2 FINDINGS CLOSED	6 APP DEFECTS STILL OPEN
--	--	---	--

The verdict

Partially aligned — and the misalignment is a coordination gap, not a disagreement.

WHAT IS GENUINELY GOOD

The new application pipelines are **better engineering than I specified**. The AI review derives its verdict deterministically from the findings rather than trusting the model's own word, runs Claude with read-only tools and an explicit deny-list so the OAuth token cannot be read out of `/proc`, treats the diff as untrusted input against prompt injection, and posts inline PR comments only on lines GitHub will actually accept. It also adds SCA and SAST scanning, which my strategy did not ask for. None of that is box-ticking.

WHERE IT DOES NOT LINE UP

The pipeline gates **the one layer the Dev team owns** — unit tests — and **none of the three QA owns**. API, UI and Data tests do not run. Of the 80 automated tests in the repo, **62 execute in CI and 18 do not**, and the 18 are the ones that cover authorization, the human-in-the-loop gates and the end-to-end journeys.

This is not a dispute about method. The pipeline work and my strategy document appear to have been written in parallel over the same few days, so the strategy was never an input to the pipeline. The fix is additive: a Postgres service container and three steps. Nothing already built needs to change.

Two of my earlier findings were independently closed, which matters — it means the loop works when the finding reaches the right person.

What changed

Seven commits on main, three on release/1.00, and a new CI branch.

New — application CI FEATURE/APP-CICD-PIPELINES

Four files, of which two are real and two are empty placeholders:

FILE	STATE	WHAT IT GATES
workbench/pr-review.yml	COMPLETE	restore, build, unit tests , webapp build, AI verdict
mock-api/pr-review.yml	COMPLETE	same, plus a <code>deploy_all.sql</code> drift check
workbench/workbench.yml	EMPTY	— build/deploy not written
mock-api/mock-api.yml	EMPTY	— build/deploy not written

Both PR pipelines trigger on PRs to `release/1.00`, path-filtered per service. That is a sensible split — Workbench changes do not run the Mock API build and vice versa.

Improved — the IaC pipeline now has real gates

My 12 September finding was that Trivy, Checkov and `terraform plan` were all soft-fail, so a PR with genuine security findings still went green and auto-applied. **Largely closed:**

STEP	BEFORE	NOW
terraform validate	—	HARD GATE
TFLint	warn only	HARD GATE on error severity
Trivy	soft-fail	HARD GATE on HIGH/CRITICAL
terraform plan / init	warn, continue	HARD GATE
Checkov	soft-fail	soft-fail — a stated decision, with every suppression justified

New infrastructure

Eight modules now wired into dev: PostgreSQL Flexible Server, Key Vault, Storage Account, Log Analytics, Container Apps, Container Apps Environment, Private DNS zone, ACR — plus container env vars and Key Vault / Storage role assignments.

DEF-008 CLOSED

The VNet is no longer orphaned. `main.cae.tf` now passes `infrastructure_subnet_id = module.vnet.subnet_ids["snet-cae-..."]` and links the private DNS zone. The delegated subnet that existed unused on 17 September is now actually doing its job.

Application changes

Swagger UI added to both services, Development-only. That is the entire `src/` diff — two `Program.cs` files and two `.csproj` files.

Alignment scorecard

My strategy of 17 September against what the pipeline actually does.

LAYER	OWNER	STRATEGY SAYS	PIPELINE DOES	TESTS	
Unit	Dev	Hard gate, every PR	Hard gate, every PR	62	ALIGNED
API	QA	Hard gate, every PR	Not run	12	GAP
Data	QA	Hard gate, every PR	Not run	0*	GAP
UI	QA	Gate on merge	Not run	6	GAP
AI evaluation	QA	Nightly, report only	Not run	12	GAP
Image build + scan	DevOps	Gate on merge	Placeholder is empty	—	GAP
Build / compile	Dev	Hard gate	Hard gate, both services + webapp	—	ALIGNED
AI code review	shared	Reuse the IaC pattern	Done, and better	—	EXCEEDED
SCA / SAST	shared	not specified	Mend + Semgrep, informational	—	BONUS

* Data-layer tests are specified (24 cases, DT-01...24) but not yet written – that one is QA's own outstanding work, not a pipeline omission.

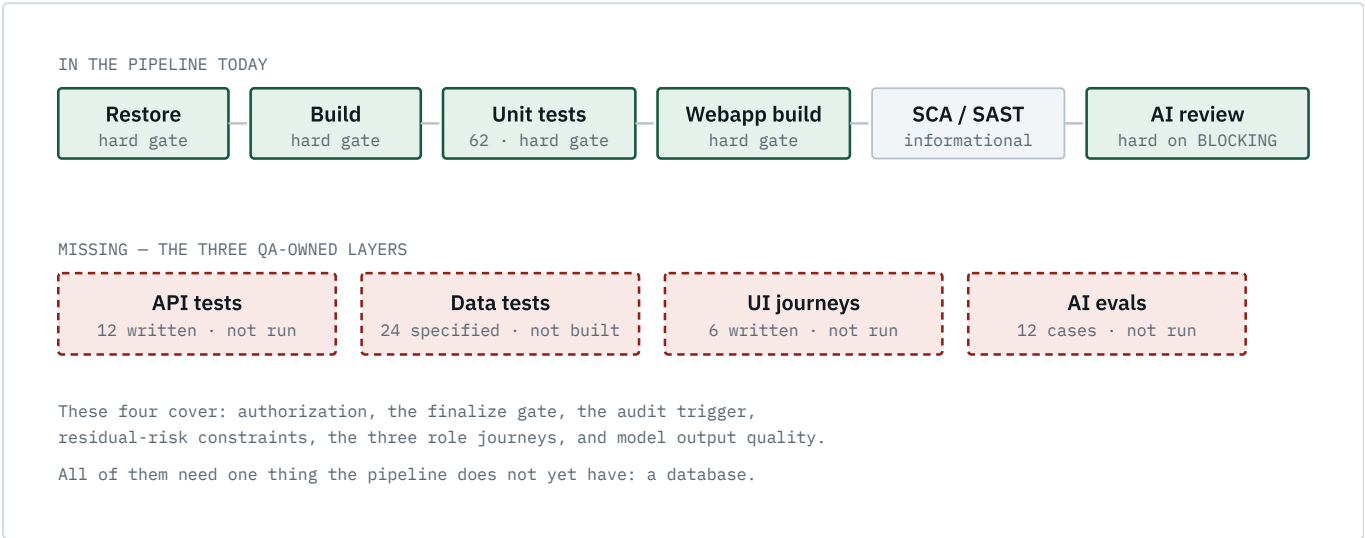


Figure 1. The gap is not scattered — it is exactly the set of tests that need a running database or a running app. Add a Postgres service container and most of it becomes reachable.

WHY THIS MATTERS MORE THAN THE NUMBERS SUGGEST

The 18 tests that do not run are not a random 18. They are the ones that prove **DEF-001, DEF-002, DEF-003, DEF-004 and DEF-006** — every open defect that has an automated reproduction. A pipeline that runs 62 of 80 tests sounds like 78% coverage; in practice it means **every known defect passes CI silently**.

The methodology

Five principles the QA suites are built on. Worth understanding before extending them, because each solves a specific way test suites go wrong.

1 · Each layer tests only what the layers below cannot

The commonest way a test strategy fails is asking one layer to cover another's ground — which doubles cost and halves clarity about what actually broke.

LAYER	ONLY IT CAN REACH	MUST NEVER DO
Unit	Business logic in isolation, in milliseconds	Touch I/O
API	Status mapping, authorization, the HTTP contract	Re-assert scoring arithmetic
Data	Triggers, CHECK constraints, transactional atomicity	Go through HTTP
UI	Whether the screens compose into a workflow	Re-prove API behaviour

The clearest example: **no endpoint anywhere attempts** `UPDATE audit_event`, so no API or UI test can ever exercise the append-only trigger. US-9.2 — the product's flagship governance claim — is reachable only from the Data layer. That is why the Data layer is not optional.

2 · Known defects are recorded, never deleted

Where the product does not yet do the right thing, the test asserts the **right** behaviour and is marked expected-to-fail. Three things follow: the board stays green while the gap is open; the moment someone fixes it the runner reports *"expected to fail, but passed"*, so **the fix announces itself**; and the requirement is never lost, which is what happens if you delete the test or weaken it to match the bug.

3 · Green means nothing until you have broken something on purpose

Before trusting a governance test, invert the rule it guards and confirm precisely that test fails — then revert. Done for the finalization gate: replacing the condition with `if (false)` failed exactly five tests and left the other seven green, because those call the readiness check directly and never traverse the gate. That selectivity is the signal. A suite that fails more, or fewer, is telling you something different from what you think.

4 · Never let a dead environment look like a pass

An expected-to-fail test passes when it fails for *any* reason — including a refused connection. So every suite first confirms its dependency is reachable and aborts loudly if not. Without that guard, running the suite against nothing produces a fully green board proving absolutely nothing. This is why the API suite has a `global-setup` that pings before anything else.

5 · Tests name the story they prove

```
test('US-8.2 AC5 — a vote is recorded against the member by name', ...)
```

Both runners emit test names into their result files, so the traceability matrix becomes a **build artifact** rather than a spreadsheet someone has to maintain. A story with no matching test name shows up as an empty row automatically. This is also what lets the new pipeline's `PublishTestResults` step do double duty — it is already publishing the data a matrix would be generated from.

And one thing deliberately not automated

Six scenarios stay manual because automation would answer the wrong question confidently — most importantly **AI narrative quality** (the eval harness checks a narrative is well-formed and cites something; it cannot judge whether the citation is apposite or the reasoning sound) and **examiner-readiness of the audit export** (US-9.1 asks for a format "suitable for handing to an examiner" — only reading it can assess that).

Closing the gap

Additive only — nothing already built needs changing.

All four missing layers need the same one thing the pipeline lacks: **a database**. Azure DevOps service containers provide it in three lines.

```
resources:
  containers:
    - container: pg
      image: postgres:16
      env: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: risk_governance_db }
      ports: [ '5432:5432' ]
```

... after the existing '.NET Unit Tests' step:

```
# HARD GATE: data layer — triggers and CHECK constraints (US-9.2, US-7.1)
- script: dotnet test tests/...IntegrationTests -c Release --logger trx
  displayName: 'Data Tests (Testcontainers)'
```

```
# HARD GATE: API + authorization (DEF-001/002/003)
- script: |
    psql -h localhost -U postgres -d risk_governance_db \
      -f src/4-Persistence/...DB/deploy_all.sql
    dotnet run --project src/1-API/... &
    cd tests/e2e && npm ci && npm run test:api
  displayName: 'API Tests (Playwright)'
```

The UI project needs the Vite dev server too, so it belongs on the merge pipeline rather than the PR one — keeping PR feedback under ten minutes, which is the design constraint. A gate slower than that gets bypassed, and a bypassed gate is worse than none.

A SECOND BENEFIT, FREE

That `psql ... deploy_all.sql` line is also the **proof-of-concept for DEF-007**. Applying the schema in CI is the same operation the release pipeline needs against the Azure database. Solve it once for tests and the deployment blocker is most of the way solved too.

Sequence, with nine days left

#	ACTION	OWNER	WHY NOW
1	Add the Postgres service container + API test step to both PR pipelines	DevOps + QA	Turns 12 written tests from shelfware into a gate; unblocks everything below
2	DEF-007 — schema deployment	DevOps + Dev	PostgreSQL is now provisioned in Azure and <i>empty</i> . The blocker got closer, not further away
3	Build the Data-layer suite (DT-01...24)	QA	The only route to US-9.2; needs no API key and no running app
4	DEF-002 / DEF-004	Dev	The two High governance defects; 15% of the score
5	Fill the empty <code>workbench.yml</code> / <code>mock-api.yml</code> — image build, scan, push	DevOps	Nothing deploys without them
6	Wire Log Analytics to the Container Apps Environment	DevOps	Module exists; one commented line from working
7	Merge <code>release/1.00</code> → <code>main</code>	DevOps	Raised 12 Sep. <code>main</code> still has no product on it

Defect status

Established by diffing each implementing file, not inferred from a test run.

ID	DEFECT	STATUS
DEF-001	User directory readable with no credential	OPEN — unchanged
DEF-002	No role enforcement; actor self-declared	OPEN — unchanged
DEF-003	Unknown ids surface as HTTP 500	OPEN — unchanged
DEF-004	Empty assessment reaches the committee	OPEN — unchanged
DEF-005	<code>control</code> table never seeded	OPEN — unchanged
DEF-006	<code>func_getOrCreateAssessment</code> race	OPEN — unchanged
DEF-007	No schema deployment path to Azure	OPEN — now more urgent
DEF-008	VNet provisioned but unused	CLOSED
DEF-009	Containers run as root	OPEN — Dockerfiles unchanged
DEF-010	Observability disabled	PARTIAL — module added, still not wired to the CAE
DEF-011	No HEALTHCHECK, floating base tags	OPEN
—	IaC scans soft-fail	CLOSED — Trivy, TFLint, validate and plan now hard-gate

DEF-007 HAS BECOME THE CRITICAL PATH

On 17 September the Azure database did not exist, so "nothing applies the schema" was a future problem. It now **does** exist — PostgreSQL Flexible Server is provisioned, with Key Vault, Storage, Container Apps and networking around it. The infrastructure is ready and *empty*, and still nothing populates it. The first deploy will start containers cleanly and fail on every request.

This also remains unowned. Open question **Q6 — who owns database schema deployment** — was raised on 17 September and has no name against it. That is precisely why it fell through the gap between two owners in the first place.

Reviewed against `origin/main` at `4e36778`, `origin/release/1.00` at `bc45271` and `origin/feature/app-cicd-pipelines` at `70f749e`, fetched 21 September 2026. Defect status established by diffing each implementing file across the range. Pipeline behaviour read from the YAML, not observed running — no build has been triggered from this review. Companion documents: *QA & BA Strategy*, *UI, API and Data Test Strategy*, *QA Test Case Register*, *Deployment Increment Review*.